9 Advanced C++ I/O

Chapter Objectives

9.1 Creating your own manipulators

9.2 File I/O basics

9.3 Unformatted, binary I/O

9.4 More unformatted I/O functions

9.5 Random access

9.6 Checking the I/O status

9.7 Customized I/O and files

THIS chapter continues the examination of the C++ I/O system. In it you will learn to create your own I/O manipulators and work with files. Keep in mind that the C++ I?O system is both rich and flexible and contains many features While it is beyond the scope of this book to include all of those features, the most important ones are discussed here. A complete description of the C++ I/O system can be found in my book *C++: The complete Reference* (Berkeley: Osborne/McGraw-Hill).

Note: *The C++ I/O system described in this chapter reflects the one defined by Standard C++ and is compatible with all major C++ compilers. If you have an older or conforming compiler, its I/O system will not have all the capabilities described here.*

Review Skills Check

Before proceeding, you should be able to correctly answer the following questions and do the exercises.

1. Write a program that displays the sentence "C++ is fun" in a 40-character-wide filed using a colon (:) as the fill character.
2. Write a program that displays the outcome of 10/3 to three decimal places. Use **ios** member functions to do this.
3. Redo the preceding program using I/O manipulators.
4. What is an inserter? What is an extractor?
5. Given the following class, create an inserter and an extractor for it.

```
class date
{
    char d[9]; // store date as string: mm/dd/yy
public:
    // add inserter and extractor
};
```

6. What header must be included if your program is to use I/O manipulators that take parameters?
7. What predefined streams are created when a C++ program begins execution?

9.1 CREATING YOUR OWN MANIPULATORS

In addition to overloading the insertion and extraction operators, you can further customize C++'s I/O system by creating your own manipulator functions. Custom manipulators are important for two main reasons. First, a manipulator can consolidate a sequence of several separate I/O operations. For example, it is not uncommon to have situations in which the same sequence of I/O operations occurs frequently within a program. In these cases you can use a custom manipulator to perform these actions, thus simplifying your source code and preventing accidental errors. Second, a custom manipulator can be important when you need to perform I/O operations no a nonstandard device. For example, you could use a manipulator to send control codes to a special type of printer or an optic recognition system.

Custom manipulators are a feature of C++ that supports OOP, but they can also benefit programs that aren't object oriented. As you will see, custom manipulators can help make any I/O-intensive program clearer and more efficient.

As you know, there are two basic types of manipulators: those that operate on input streams and those that operate on output streams. In addition to these two broad categories, there is a secondary division: those manipulators that take an argument and those that don't. There are some significant differences between the way a parameterless manipulator and a parameterized manipulator are created. Further, creating parameterized manipulators is substantially more difficult then creating parameterless ones and is beyond the scope of this book. However, writing your own parameterless manipulators is quite easy and is examined here.
All parameterless manipulator output functions have this skeleton:

```
ostream &manip_name(ostream &stream)
{
    // your code here
    return stream;
}
```

Here *manip-name* is the name of the manipulator and *stream* is a reference to the invoking stream. A reference to the stream is returned. This is necessary if a manipulator is used as part of a larger I/O expression. It is important to understand that even though the manipulator has as its single argument a reference to the stream upon which it is operating, no argument is used when the manipulator is called in an output operation.
All parameterless input manipulator functions have this skeleton:

```
istream &manip_name(istream & stream)
{
    // your code here
    return stream;
}
```

An input manipulator receives a reference to the stream on which it was invoked. This stream must be returned by the manipulator.
Remember: *It is crucial that your manipulators return a reference to the invoking stream. If this is not done, your manipulators cannot be used in sequence of input or output operations.*

EXAMPLES

1. As a simple first example, the following program creates a manipulator called **setup()** that sets the filed width to 10, the precision to 4, and the fill character to *.

```
#include <iostream>
using namespace std;

ostream &setup(ostream &stream)
{
    stream.width(10);
    stream.precision(4);
    stream.fill('*');

    return stream;
}

int main()
{
    cout << setup << 123.123456;
```

```
    return 0;
}
```

2. Custom manipulators need not be complex to be useful. For example, the simple manipulators **atn()** and **note()** show here, provide a shorter way to output frequently used words or phrases.

```
#include <iostream>
using namespace std;

// Attention:
ostream &atn(ostream &stream)
{
    stream << "Attention: ";
    return stream;
}

// Please note:
ostream &note(ostream &stream)
{
    stream << "Please Note: ";
    return stream;
}

int main()
{
    cout << atn << "High voltage circuit\n";
    cout << note << "Turn off all lights\n";

    return 0;
}
```

Even though they are simple, if used frequently, these manipulators save you from some tedious typing.

3. This program creates the **getpass()** input manipulator, which rings the bell and then prompts for a password:

```
#include <iostream>
#include <cstring>
using namespace std;

// A simple input manipulator
istream &getpass(istream &stream)
{
    cout << '\a'; // sound bell
    cout << "Enter password: ";
    return stream;
}

int main()
{
    char pw[80];
```

```
    do
    {
        cin >> getpass >> pw;
    }
    while(strcmp(pw, "password"));

    cout << "Logon complete\n";

    return 0;
}
```

EXERCISES

1. Create an output manipulator that displays the current system time and date. Call this manipulator **td()**.
2. Create an output manipulator called **sethex()** that sets output to hexadecimal and turns on the **uppercase** and **showbase** flags. Also, create an output manipulator called **reset()** that undoes the changes made by **sethex()**.
3. Create an input manipulator called **skipchar()** that reads and ignores the next ten characters from the input stream.

9.2 FILE I/O BASICS

It is now time to turn our attention to file I/O. As mentioned in the preceding chapter, file I/O and console I/O are closely related. In fact, the same class hierarchy that supports console I/O also supports file I/O. Thus, most of what you have already learned about I/O applies to files as well. Of course, file handling makes use of several new features.
To perform file I/O, you must include the header **<fstream>** in your program. It defines several classes, including **ifstream**, **ofstream**, and **fstream** also have access to all operations defined by **ios** (discussed in the preceding chapter).
In C++, a file is opened by linking it to a stream. There are three types of streams: input, output, and input/output. Before you can open a file, you must first obtain a stream. To create an input stream, declare an object of type **ifstream**. To create an output stream, declare an object of type **ofstream**. Streams that will be performing both input and output operations must be declared as objects of type type **fstream**. For example, this fragment creates one input stream, one output stream, and one stream capable of both input and output:

```
ifstream in;  // input
ofstream out; // output
fstream io;   // input and output
```

Once you have created a stream, one way to associate it with a file is by using the function **open()**. This function is a member of each of the three file stream classes. The prototype for each is shown here:

```
void ifstream::open(const char *filename,
                    openmode mode=ios::in);

void ofstream::open(const char *filename,
                    openmode mode=ios::out | ios::trunc);
```

```
void fstream::open(const char *filename,
                   openmode mode=ios::in | ios::out);
```

Here *filename* is the name of the file, which can include a path specifier. The value of *mode* determines how the file is opened. It must be a value of type **openmode**, which is an enumeration defined by **ios** that contains the following values:

```
ios::app
ios::ate
ios::binary
ios::in
ios::out
ios::trunc
```

You can combine two or more of these values by ORing them together. Let's see what each of these values means.

Including **ios::app** causes all output to that file to be appended to the end. This value can be used only with files capable of output. Including **ios::ate** causes a seek to the end of the file to occur when the file is opened. Although **ios::ate** causes a seek to end-of-file, I/O operations can still occur anywhere within the file.

The **ios::in** value specifies that the file is capable of input. The **ios::out** value specifies that the file is capable of output.

The **ios::binary** value causes a file to be opened in binary mode. By default, all files are opened in text mode. In text mode, various character translations might take place, such as carriage return/linefeed sequences being converted into newlines. However, when a file is opened in binary mode, no such character translations will occur. Keep in mind that any file, whether it contains formatted text or raw data, can be opened in either binary or text mode. The only difference is whether character translations take place.

The **ios::trunc** value causes the contents of a preexisting file by the same name to be destroyed and the file to be truncated to zero length. When you create an output stream using **ofstream**, any preexisting file with the same name is automatically truncated.

The following fragment opens an output file called **test**:

```
ofstream mystream;
mystream.open("test");
```

Since the *mode* parameter to **open()** defaults to a value appropriate to the type of stream being opened, there is no need to specify its value in the preceding example.

If **open()** fails, the stream will evaluate to false when used in a Boolean expression. You can make use of this fact to confirm that the open operation succeeded by using a statement like this:

```
if(!mystream)
{
    cout << "Cannot open file.\n";
    // handle error
}
```

In general, you should always check the result of a call to **open()** before attempting to access the file.

You can also check to see if you have successfully opened a file by using the **is_open()** function, which is a member of **fstream**, **ifstream**, and **ofstream**. It has this prototype.

```
bool is_open();
```

It returns true if the stream is linked to an open file and false otherwise. For example, the following checks if **mystream** is currently open:

```
if(!mystream.is_open())
{
    cout << "File is not open.\n";
    // ..
```

Although it is entirely proper to open a file by using the **open()** function, most of the time you will not do so because the **ifstream**, **ofstream**, and **fstream** classes have constructor functions that automatically open the file. The constructor functions have the same parameters and defaults as the **open()** function. Therefore, the most common way you will see a file opened is shown in this example:

```
ifstream mystream("myfile"): // open file for input
```

As stated, if for some reason the file cannot be opened, the stream variable will evaluate as false when used in a conditional statement. Therefore, whether you use a constructor function to open the file or an explicit call to **open()**, you will want to confirm that the file has actually been opened by testing the value of the stream.
To close a file, use the member function **close()**. For example, to close the file linked to a stream called **mystream**, use this statement:

```
mystream.close();
```

The **close()** function takes no parameters and returns no value.
You can detect when the end of an input file has been reach by using the **eof()** member function of **ios**. It has this prototype:

```
bool eof();
```

It returns true when the end of file has been encountered and false otherwise.
Once a file has been opened, it is very easy to read textual data from it or write formatted, textual data to it. Simply use the << and >> operators the same way you do when performing console I/O, except that instead of using **cin** and **cout**, substitute a stream that is liked to a file. In a way, reading and writing files by using >> and << is like using C's **fprintf()** and **fscanf()** functions. All information is stored in the file in the same format it would be in if displayed on the screen. Therefore, a file produced by using << is a formatted text file. Typically, files that contain formatted text that you operate on using the >> and << operators should be opened for text rather than binary mode. Binary mode is best used on unformatted files, which are described later in this chapter.

EXAMPLES

1. Here is a program that creates an output file, write information to it, closes the file and opens it again as an input file, and reads in the information:

```
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    ofstream fout("test"); // create output file

    if(!fout)
    {
```

```
        cout << "Cannot open output file.\n";
        return 1;
    }

    fout << "Hello!\n";
    fout << 100 << ' ' << hex << 100 << endl;

    fout.close();

    ifstream fin("test"); // open input file

    if(!fin)
    {
        cout << "Cannot open input file.\n";
        return 1;
    }

    char str[80];
    int i;

    fin >> str >> i;
    cout << str << ' ' << i << endl;

    fin.close();

    return 0;
}
```

After you run this program, examine the contents of **test**. It will contain the following:

```
Hello!
100 64
```

As stated earlier, when the << and >> operators are used to perform file I/O, information is formatted exactly as it would appear on the screen.

2. Following is another example of disk I/O. This program reads strings entered at the keyboard and writes them to disk. The program stops when the user enters a **$** as the first character in a string. To use the program, specify the name of the output file on the command line.

```
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char *argv[])
{
    if(argc!=2)
    {
        cout << "Usage: WRITE <filename>\n";
        return 1;
    }
    ofstream out(argv[1]); // output file
```

```
    if(!out)
    {
        cout << "Cannot open output file.\n";
        return 1;
    }

    char str[80];
    cout << "Write strings to disk, '$' to stop\n";

    do
    {
        cout << ": ";
        cin >> str;
        out << str << endl;
    }
    while(*str != '$');

    out.close();
    return 0;
}
```

3. Following is a program that copies a text file and, in the process, converts all spaces into | symbols. Notice how **eof()** is used to check for the end of input file. Notice also how the input stream **fin** has its **skipws** flag turned off. This prevents leading spaces from being skipped.

```
// Convert spaces to |s.
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char *argv[])
{
    if(argc!=3)
    {
        cout << "Usage: CONVERT <input> <output>\n";
        return 1;
    }

    ifstream fin(argv[1]); // open input file
    ofstream fout(argv[2]); // create output file

    if(!fout)
    {
        cout << "Cannot open output file.\n";
        return 1;
    }

    if(!fin)
    {
        cout << "Cannot open input file.\n";
        return 1;
```

```
    }

    char ch;

    fin.unsetf(ios::skipws); // do not skip spaces
    while(!fin.eof())
    {
        fin >> ch;
        if(ch==' ')
            ch = '|';
        if(!fin.eof())
            fout << ch;
    }

    fin.close();
    fout.close();

    return 0;
}
```

4. There are a few differences between C++'s original I/O library and the modern Standard C++ library that you should be aware of, especially if you are converting older code. First, in the original I/O library, **open()** allowed a third parameter, which specified the file's protection mode. This parameter defaulted to a normal file. The modern I/O library does not support this parameter.

   Second, when you are using the old library to open a stream for input and output using **fstream**, you must explicitly specify both the **ios::in** and the **ios::out** *mode* values. No default value for *mode* is supplied. This applies to both the **fstream** constructor and to its **open()** function. For example, using the old I/O library you must use a call to **open()** as shown here to open a file for input and output:

```
fstream mystream;
mystream.open("test", ios::in | ios::out);
```

   In the modern I/O library, an object of type **fstream** automatically opens files for input and output when the *mode* parameter is not supplied.

   Finally, in the old I/O system, the *mode* parameter could also include either **ios::nocreate()**, which causes the **open()** function to fail if the file does not already exist, or **ios::noreplace**, which causes the **open()** function to fail if the file does not already exist. These values are not supported by Standard C++.

EXERCISES

1. Write a program that will copy a text file. Have this program count the number of characters copied and display this result. Why does the number displayed differ from that shown when you list the output file in the directory?

2. Write a program that writes the following table of information to a file called **phone**:

   `Isaac Newton, 415 555-3423`

   `Robert Goddard, 213 555-2312`

   `Enrico Fermi, 202 555-1111`

3. Write a program that counts the number of words in a file. For simplicity, assume that anything surrounded by whitespace is a word.

4. What does **is_open()** do?

9.3 UNFORMATTED, BINARY I/O

Although formatted text files such as those produced by the preceding examples are useful in a variety of situations, they do not have the same binary representation of the data as that used internally by your program rather than the human-readable text that data is translated into by the << and >> operators. Thus, unformatted functions give you detailed control over how files are written and read.
The lowest-level unformatted I/O functions are **get()** and **put()**. You can read a byte by using **get()** and write a byte by using **put()**. These functions are members of all input and output stream classes, respectively. The **get()** function has many forms, but the most commonly used version is shown here, along with **put()**:

```
istream &get(char &ch);
ostream &put(char &ch);
```

The **get()** function reads a single character from the associated stream and puts that value in *ch*. It returns a reference to the stream. If a read is attempted at end-of-file, on return the invoking stream will evaluate to false when used in an expression. The **put()** function writes *ch* to the stream and returns a reference to the stream.
To read and write blocks of data, use the **read()** and **write()** functions, which are also members of the input and output stream classes, respectively. Their prototypes are shown here:

```
istream &read(char *buf, streamsize num);
ostream &write(const char *buf, streamsize num);
```

The **read()** function reads *num* bytes from the invoking stream and puts them in the buffer pointed to by *buf*. The **write()** function writes *num* bytes to the associated stream from the buffer pointed to by *buf*. The **streamsize** type is some form of integer. An object of type **streamsize** is capable of holding the largest number of bytes that will be transferred in any one I/O operation.
If the end of the file is reached before *num* characters have been read, **read()** simply stops, and the buffer contains as many characters as were available. You can find out how many characters have been read by using the member function **gcount()**, which has this prototype:

```
streamsize gcount();
```

It returns the number of characters read by the last unformatted input operations.
When you are using the unformatted file functions, most often you will open a file for binary rather than text operations. The reason for this is easy to understand: specifying **ios::binary** prevents any character translations from occurring. This is important when the binary representations of data such as integers, **float**, and pointers are stored in the file. However, it is perfectly acceptable to use the unformatted functions on a file opened in text mode-as long as that file actually contains only text. But remember, some character translations may occur.

EXAMPLES

1. The next program will display the contents of any file on the screen. It uses the **get()** function.

```
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char *argv[])
{
    char ch;

    if(argc!=2)
    {
        cout << "Usage: PR <filename>\n";
        return 1;
    }

    ifstream in(argv[1], ios::in | ios::binary);
    if(!in)
    {
        cout << "Cannot open file.\n";
        return 1;
    }

    while(!in.eof())
    {
        in.get(ch);
        cout << ch;
    }

    in.close();

    return 0;
}
```

2. This program uses **put()** to write characters to a file until the user enters a dollar sign:

```
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char *argv[])
{
    char ch;

    if(argc!=2)
    {
        cout << "Usage: WRITE <filename>\n";
        return 1;
    }

    ofstream out(argv[1], ios::out | ios::binary);
    if(!out)
    {
        cout << "Cannot open file.\n";
```

```
        return 1;
    }

    cout << "Enter a $ to stop\n";
    do
    {
        cout << ": ";
        cin.get(ch);
        out.put(ch);
    }
    while(ch!='$');

    out.close();

    return 0;
}
```

Notice that the program uses **get()** to read characters from **cin**. This prevents leading spaces from being discarded.

3. Here is a program that uses **write()** to write a **double** and a string to a file called **test**:

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main()
{
    ofstream out("test", ios::out | ios::binary);

    if(!out)
    {
        cout << "Cannot open output file.\n";
        return 1;
    }

    double num = 100.45;
    char str[] = "This is a test";

    out.write((char *) &num, sizeof(double)) ;
    out.write(str, strlen(str));

    out.close();

    return 0;
}
```

Note: *The type cast to* ***(char *)*** *inside the call to* ***write()*** *is necessary when outputting a buffer that is not defined as a character array. Because of C++'s strong type checking, a pointer of one type will not automatically be converted into a pointer of another type.*

4. This program uses **read()** to read the file created by the program in Example 3:

```
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    ifstream in("test", ios::in | ios::binary);

    if(!in)
    {
        cout << "Cannot open input file.\n";
        return 1;
    }

    double num;
    char str[80];

    in.read((char *) &num, sizeof(double)) ;
    in.read(str, 14);
    str[14] = '\0'; // null terminate str

    cout << num << ' ' << str;

    in.close();

    return 0;
}
```

As is the case with the program in the preceding example, the type cast inside **read()** is necessary because C++ will not automatically convert a pointer of one type to another.

5. The following program first writes an array of **double** values to a file and then reads them back. It also reports the number of characters read.

```
// Demonstrate gcount()
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    ofstream out("test", ios::out | ios::binary);

    if(!out)
    {
        cout << "Cannot open output file.\n";
        return 1;
    }

    double nums[4] = {1.1, 2.2, 3.3, 4.4};

    out.write((char *) nums, sizeof(nums));
    out.close();
```

```
    ifstream in("test", ios::in | ios::binary);

    if(!in)
    {
        cout << "Cannot open input file.\n";
        return 1;
    }

    in.read((char *) &nums, sizeof(nums)) ;

    int i;
    for(i=0; i<4; i++)
        cout << nums[i] << ' ';

    cout << '\n';

    cout << in.gcount() << " characters read\n";
    in.close();

    return 0;
}
```

EXERCISES

1. Rewrite your answers to Exercises 1 and 3 in the preceding section (Section 9.2) so that they use **get()**, **put()**, **read()**, and/or **write()**. (Use whichever of these functions you deem most appropriate.)

2. Given the following class, write a program that outputs the contents of the class to a file. Create an inserter function for this purpose.

```
class account
{
    int custnum;
    char name[80];
    double balance;
public:
    account(int c, char *n, double b)
    {
        custom = c;
        strcpy(name, n);
        balance = b;
    }
    // create inserter here
};
```

9.4 MORE UNFORMATTED I/O FUNCTIONS

In addition to the form shown earlier, there are several different ways in which the **get()** function is overloaded. The prototypes for the three most commonly used overloaded forms are shown here:

```
istream &get(char *buf, streamsize num);
istream &get(char *buf, streamsize num, char delim);
int get();
```

The first form reads characters into the array pointed to by *buf* until either *num*-1 characters have been read, a newline is found, or the end of the file has been encountered. They array pointed to by *buf* will be null terminated by **get()**. If the newline character is encountered in the input stream, it is *not* extracted. Instead, it remains in the stream until the next input operation.
The second form reads characters into the array pointed to by *buf* until either *num*-1 characters have been read, the character specified by *delim* has been found, or the end of the file has been encountered. The array pointed to by *buf* will be null terminated by **get()**. If the delimiter character is encountered in the input stream, it is *not* extracted. Instead, it remains in the stream until the next input operation.
The third overloaded form of **get()** returns the next character from the stream. It returns EOF if the end of the file is encountered. This form of **get()** is similar to C's **getc()** function.
Another function that performs input is **getline()**. It is a member of each input stream class. Its prototype are shown here:

```
istream &getline(char *buf, streamsize num);
istream &getline(char *buf, streamsize num, char delim);
```

The first form reads characters into the array pointed to by *buf* until either *num*-1 characters have been read, a newline character has been found, or the end of the file has been encountered. The array pointed to by *buf* will be null terminated by **getline()**. If the newline character is encountered in the input stream, it is extracted, but it is not put into *buf*.
The second form reads characters into the array pointed to by *buf* until either *num*-1 characters have been read, the character specified by *delim* has been found, or the end of the file has been encountered. The array pointed to by *buf* will be null terminated by **getline()**. If the delimiter character is encountered in the input stream, it is extracted, but it is not put into *buf*.
As you can see, the two versions of **getline()** are virtually identical to the **get(***buf*, *num***)** and **get(***buf*, *num*, *delim***)** versions of **get()**. Both read characters from input and put them into the array pointed to by *buf* until either *num*-1 characters have been read or until the delimiter character or the end of the file is encountered. The difference between **get()** and **getline()** is that **getline()** reads and removes the delimiter from the input stream; **get()** does not.
You can obtain the next character in the input stream without removing it from that stream by using **peek()**. This function is a member of the input stream classes and has this prototype:

```
int peek();
```

It returns the next character in the stream; it returns EOF if the end of the file is encountered.
You can return the last character read from a stream to that stream by using **putback()**, which is a member of the input stream classes. Its prototype is shown here:

```
istream &putback(char c);
```

where *c* is the last character read.
When output is performed, data is not immediately written to the physical device linked to the stream. Instead, information is stored in an internal buffer until the buffer is full. Only then are the contents of that buffer written to disk. However, you can force the information to be physically written to disk before the buffer is full by calling **flush()**. **flush()** is a member of the output stream classes and has this prototype:

```
ostream &flush();
```

Calls to **flush()** might be warranted when a program is going to be used in adverse environments (in situations where power outages occur frequently, for example).

EXAMPLES

1. As you know, when you use >> to read a string, it stops reading when the first whitespace character is encountered. This makes it useless for reading a string containing spaces. However, you can overcome this problem by using **getline()**, as this program illustrates:

```
// Use getline() to read a string that contains spaces.
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    char str[80];

    cout << "Enter your name: ";
    cin.getline(str, 79);

    cout << str << '\n';

    return 0;
}
```

   In this program, the delimited used by **getline()** is the newline. This makes **getline()** act much like the standard **gets()** function.

2. In real programming situations, the functions **peek()** and **putback()** are especially useful because they let you more easily handle situations in which you do not know what type of information is being input at any point in time. The following program gives the flavor of this. It reads either strings or integers from a file. The strings and integers can occur in any order.

```
// Demonstrate peek()
#include <iostream>
#include <fstream>
#include <cctype>
#include <cstdlib>
using namespace std;

int main()
{
    char ch;
    ofstream out("test", ios::out | ios::binary);

    if(!out)
    {
        cout << "Cannot open output file.\n";
        return 1;
    }

    char str[80], *p;

    out << 123 << "this is a test" << 23;
    out << "Hello there!" << 99 << "sdf" << endl;
```

```
    out.close();

    ifstream in("test", ios::in | ios::binary);

    if(!in)
    {
        cout << "Cannot open input file.\n";
        return 1;
    }

    do
    {
        p = str;
        ch = in.peek(); // see what type of char is next
        if(isdigit(ch))
        {
            while(isdigit(*p=in.get())) // read integer
                p++;
            in.putback(*p); // return char to stream
            *p = '\0'; // null-terminate the string
            cout << "Integer: " << atoi(str);
        }
        else if(isalpha(ch))
        {
            while(isalpha(*p=in.get())) // read a string
                p++;
            in.putback(*p);
            *p = '\0';
            cout << "String: " << str;
        }
        else
            in.get(); // ignore
        cout << '\n';
    }
    while(!in.eof());

    in.close();
    return 0;
}
```

EXERCISES

1. Rewrite the program in Example 1 so it uses **get()** instead of **getline()**. Does the program function differently?

2. Write a program that reads a text file one line at a time and displays each line on the screen. Use **getline()**.

3. On your own, think about why there may be cases in which a call to **flush()** is appropriate.

9.5 RANDOM ACCESS

In C++'s I/O system, you perform random access by using the **seekg()** and **seekp()** functions which are members of the input and output stream classes, respectively. Their most common forms are shown here:

```
istream &seekg(off_type offset, seekdir origin);
ostream &seekp(off_type offset, seekdir origin);
```

Here **off_tyoe** is an integer type defined by **ios** that is capable of containing the largest valid value that *offset* can have. **seekdir** is an enumeration defined by **ios** that has these values:

Value	Meaning
ios::beg	Seek from beginning
ios::cur	Seek from current location
ios::end	Seek from end

The C++ I/O system manages two pointers associated with a file. One is the *get pointer*, which specifies where in the file the next input operation will occur. The other is the *put pointer*, which specifies where in the file the next output operation will occur. Each time an input or output operation takes place, the appropriate pointer is automatically sequentially advanced. However, by using the **seekg()** and **seekp()** functions, it is possible to access the file in a non-sequential fashion.

The **seekg()** function moves the associated file's current get pointer *offset* number of bytes from the specified *origin*. The **seekp()** function moves the associated file's current put pointer *offset* number of bytes from the specified *origin*.

In general, files that will be accessed via **seekg()** and **seekp())** should be opened for binary file operations. This prevents character translations from occurring which may affect the apparent position of an item within a file.

You can determine the current position of each file pointer by using these member functions:

```
pos_type tellg();
pos_type tellp();
```

Here **pos_type** is an integer type defined by **ios** that is capable of holding the largest value that defines a file position.

There are overloaded versions of **seekg()** and **seekp()** that move the file pointers to the location specified by the return values of **tellg()** and **tellp()**. Their prototypes are shown here:

```
istream &seekg(pos_type position);
ostream &seekp(pos_type position);
```

EXAMPLES

1. The following program demonstrates the **seekp()** function. It allows you to change a specific character in a file. Specify a file name on the command line, followed by the number of the byte in the file you want to change, followed by the new character. Notice that the file is opened for read/write operations.

```
#include <iostream>
#include <fstream>
#include <cstdlib>
using namespace std;

int main(int argc, char *argv[])
```

```
{
    if(argc!=4)
    {
        cout << "Usage: CHANGE <filename> <byte> <char>\n";
        return 1;
    }

    fstream out(argv[1], ios::in | ios::out | ios::binary);

    if(!out)
    {
        cout << "Cannot open file.\n";
        return 1;
    }

    out.seekp(atoi(argv[2]), ios::beg);

    out.put(*argv[3]);
    out.close();

    return 0;
}
```

2. The next program uses **seekg()** to position the get pointer into the middle of a file and then displays the contents of that file from that point. The name of the file and the location to begin reading from are specified on the command line.

```
// Demonstrate seekg()
#include <iostream>
#include <fstream>
#include <cstdlib>
using namespace std;

int main(int argc, char *argv[])
{
    char ch;

    if(argc!=3)
    {
        cout << "Usage: LOCATE <filename> <loc>\n";
        return 1;
    }

    ifstream in(argv[1], ios::in | ios::binary);

    if(!in)
    {
        cout << "Cannot open input file.\n";
        return 1;
    }

    in.seekg(atoi(argv[2]), ios::beg);
```

```
    while(!in.eof())
    {
        in.get(ch);
        cout << ch;
    }

    in.close();

    return 0;
}
```

EXERCISES

1. Write a program that displays a text file backwards. Hint: Think about this before creating your program. The solution is easier than you might imagine.
2. Write a program that swaps each character pair in a text file. For example, if the file contains "1234", then after the program is run, the file will contain "2143". (For simplicity, you may assume that the file contains an even number of characters.)

9.6 CHECKING THE I/O STATUS

The C++ I/O system maintains status information about the outcome of each I/O operation. The current status of an I/O stream is described in an object of type **iostate**, which is an enumeration defined by **ios** that includes these members:

Name	Meaning
goodbit	No errors occurred.
eofbit	End-of-file has been encountered.
failbit	A nonfatal I/O error has occurred.
badbit	A fatal I/O error has occurred.

For older compilers, the I/O status flags are held in an **int** rather than an object of type **iostate**. There are two ways in which you can obtain I/O status information. First, you can call the **rdstate()** function, which is a member of **ios**. It has this prototype:

```
iostate rdstate();
```

It returns the current status of the error flags. As you can probably guess from looking at the preceding list of flags, **rdstate()** returns **goodbit** when no error has occurred. Otherwise, an error flag is returned.
The other way you can determine whether an error has occurred is by using one or more of these **ios** member functions:

```
bool bad();
bool eof();
bool fail();
bool good();
```

The **eof()** function was discussed earlier. The **bad()** function returns true if **badbit** is set. The **fail()** function returns true if **failbit** is set. The **good()** function returns true if there are no errors. Otherwise they return false.
Once an error has occurred, it might need to be cleared before your program continues. To do this, use the **ios** member function **clear()** whose prototype is shown here:

```
void clear(iostate flags=ios::goodbit);
```

If *flags* is **goodbit** (as it is by default). all error flags are cleared. Otherwise, set *flags* to the settings you desire.

EXAMPLES

1. The following program illustrates **rdstate()**. It displays the contents of a text file. If an error occurs, the function reports it by using **checkstatus()**.

```
#include <iostream>
#include <fstream>
using namespace std;

void checkstatus(ifstream &in);

int main(int argc, char *argv[])
{
    if(argc!=2)
    {
        cout << "Usage: DISPLAY <filename>\n";
        return 1;
    }

    ifstream in(argv[1]);

    if(!in)
    {
        cout << "Cannot open input file.\n";
        return 1;
    }

    char c;
    while(in.get(c))
    {
        cout << c;
        checkstatus(in);
    }

    checkstatus(in); // check final status
    in.close();

    return 0;
}

void checkstatus(ifstream &in)
{
    ios::iostate i;

    i = in.rdstate();

    if(i & ios::eofbit)
        cout << "EOF encountered\n";
```

```
    else if(i & ios::failbit)
        cout << "Non-Fatal I/O error\n";
    else if(i & ios::badbit)
        cout << "Fatal I/O error\n";
}
```

The preceding program will always report at least one "error." After the **while** loop ends, the final call to **checkstatus()** reports, as expected, that an EOF has been encountered.

2. This program displays a text file. It uses **good()** to detect a file error:

```
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char *argv[])
{
    char ch;

    if(argc!=2)
    {
        cout << "Usage: PR <filename>\n";
        return 1;
    }

    ifstream in(argv[1]);

    if(!in)
    {
        cout << "Cannot open input file.\n";
        return 1;
    }

    while(!in.eof())
    {
        in.get(ch);
        // check for error
        if(!in.good() && !in.eof())
        {
            cout << "I/O Error...terminating\n";
            return 1;
        }
        cout << ch;
    }

    in.close();

    return 0;
}
```

EXERCISES

1. Add error checking to your answers to the exercise from the preceding section.

9.7 CUSTOMIZED I/O AND FILES

In the preceding chapter, you learned how to overload the insertion and extraction operators relative to your own classes. In that chapter, only console I/O was performed. However, because all C++ streams are the same, the same overloaded inserter function, for example, can be used to output to the screen or to a file with no changes whatsoever. This is one of the most important and useful features of C++'s approach to I/O.
As stated in the previous chapter, overloaded inserters and extractors, as well as I/O manipulators, can be used with any stream as long as they are written in a general manner. If you "hard-code" a specific stream into an I/O function, its use is, of course, limited to only that stream. This is why you were urged to generalize your I/O functions whenever possible.

EXAMPLES

1. In the following program, the **coord** class overloads the << and >> operators. Notice that you can use the operator functions to write both to the screen and to a file.

```
#include <iostream>
#include <fstream>
using namespace std;

class coord
{
    int x, y;
public:
    coord(int i, int j) { x = i; y = j; }
    friend ostream &operator<<(ostream &stream, coord ob);
    friend istream &operator>>(istream &stream, coord &ob);
};

ostream &operator<<(ostream &stream, coord ob)
{
    stream << ob.x << ' ' << ob.y << '\n';

    return stream;
}

istream &operator>>(istream &stream, coord &ob)
{
    stream >> ob.x >> ob.y;

    return stream;
}

int main()
{
    coord o1(1, 2), o2(3, 4);
    ofstream out("test");

    if(!out)
    {
        cout << "Cannot open output file.\n";
```

```
        return 1;
    }

    out << o1 << o2;

    out.close();

    ifstream in("test");

    if(!in)
    {
        cout << "Cannot open input file.\n";
        return 1;
    }

    coord o3(0, 0), o4(0, 0);
    in >> o3 >> o4;

    cout << o3 << o4;

    in.close();

    return 0;
}
```

2. All of the I/O manipulators can be used with files. For example, in this reworked version of a program presented earlier in this chapter, the same manipulator that writes to the screen will also write to a file:

```
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;

// Attention:
ostream &atn(ostream &stream)
{
    stream << "Attention: ";
    return stream;
}

// Please note:
ostream &note(ostream &stream)
{
    stream << "Please Note: ";
    return stream;
}

int main()
{
     ofstream out("test");
```

```
    if(!out)
    {
        cout << "Cannot open output file.\n";
        return 1;
    }

    // write to screen
    cout << atn << "High voltage circuit\n";
    cout << note << "Turn off all lights\n";

    // write to file
    out << atn << "High voltage circuit\n";
    out << note << "Turn off all lights\n";

    out.close();

    return 0;
}
```

EXERCISE

1. On your own, experiment with the programs from the preceding chapter, trying each on a disk file.

SKILLS CHECK

Mastery Skills Check

At this point you should be able to perform the following exercises and answer the questions.

1. Create an output manipulator that outputs three tabs and then sets the field width to 20. Demonstrate that your manipulator works.
2. Create an input manipulator that reads and discards all non-alphabetical characters. When the first alphabetical character is read, have the manipulator return it to the input stream and return. Call this manipulator **findalpha**.
3. Write a program that copies a text file. In the process, reverse the case of all letters.
4. Write a program that reads a text file and then reports the number of times each letter in the alphabet occurs in the file.
5. If you have not done so, add complete error checking to your solutions to Exercises 3 and 4 above.
6. What function positions the get pointer? What function positions the put pointer?

Cumulative Skills Check

This section checks how well you have integrated material in this chapter with that from the preceding chapters.

1. Following is a reworked version of the **inventory** class presented in the preceding chapter. Write a program that fills in the functions **store()** and **retrieve()**. Next, create a small inventory file on disk containing a few entries. Then, using random I/O, allow the user to display the information about any item by specifying its record number.

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

#define SIZE 40

class inventory
{
    char item[SIZE]; // name of item
    int onhand; // number on hand
    double cost; // cost of item
public:
    inventory(char *i, int o, double c)
    {
        strcpy(item, i);
        onhand = o;
        cost = c;
    }
    void store(fstream &stream);
    void retrieve(fstream &stream);
    friend ostream &operator<<(ostream &stream, inventory ob)
       ;
    friend istream &operator>>(istream &stream, inventory &ob
       );
};

ostream &operator<<(ostream &stream, inventory ob)
{
    stream << ob.item << ": " << ob.onhand;
    stream << " on hand at $" << ob.cost << '\n';

    return stream;
}

istream &operator>>(istream &stream, inventory &ob)
{
    cout << "Enter item name: ";
    stream >> ob.item;
    cout << "Enter number on hand: ";
    stream >> ob.onhand;
    cout << "Enter cost: ";
    stream >> ob.cost;
```

```
    return stream;
}
```

2. As a special challenge, on your own, create a **stack** class for characters that stores them in a disk file rather than in an array in memory.